GIACENZE CRYPTO

monitoraggio e fiscalità delle cripto-attività

MANUALE

Configurazione JSON per le importazioni personalizzate

Documentazione di Giacenze Crypto

DOCUMENTO	creazione-json-importazioni-personalizzate.pdf
SITO	lucapasselli.github.io/GiacenzeCrypto/documentazione
GENERATO IL	14/11/2026
VERSIONE	Giacenze Crypto 1.0.66.06

AVVERTENZA

I documenti ottenuti e le informazioni presenti hanno sempre valenza informativa e meramente indicativa ed esemplificativa, e non sono in alcun modo sostitutive di una consulenza fiscale.

Questa guida spiega come creare un file JSON per importare qualunque file CSV nella raccolta dei movimenti crypto. Il JSON descrive:

- dove si trovano i dati nel CSV;
- come interpretarli (date, segni, causali);
- come trasformarli (rinomina monete, pulizia nomi, wallet).

Ogni proprietà ha un valore predefinito. Se non la inserisci nel JSON, viene usato il default indicato.

Le stesse regole sono pubblicate anche come **schema JSON**, un file che i programmi sanno leggere: configurazione-import.schema.json. Serve a un editor per segnalare gli errori mentre scrivi e a un'IA per non inventare chiavi o valori (vedi la sezione seguente).

Scrivere la configurazione con un'IA

Un assistente come ChatGPT, Claude o Gemini può scrivere la configurazione al posto tuo, ma senza le informazioni giuste tende a inventare chiavi che il programma non conosce, a sbagliare gli indici delle colonne o a scegliere tipi di movimento inesistenti. Il risultato si importa senza errori evidenti e classifica male i movimenti. Per ridurre il rischio dagli queste quattro cose.

1. **Lo schema JSON**, allegato come file oppure con il suo indirizzo, se l'assistente può leggere le pagine web: `https://lucapasselli.github.io/GiacenzeCrypto/schema/configurazione-import.schema.json`.
2. **Questo manuale**. In un solo file di testo, insieme agli altri manuali, si trova a `https://lucapasselli.github.io/GiacenzeCrypto/llms-full.txt`.
3. **La configurazione ufficiale più simile** al tuo file, presa dalla cartella `config/import/` del programma oppure da GitHub. Partire da un file che funziona è molto più sicuro che partire da zero.
4. **L'intestazione e 20-30 righe del tuo CSV**, scelte in modo che compaiano tutte le causali presenti nel file. Prima di incollarle sostituisci i dati che non vuoi condividere (identificativo utente, indirizzi dei wallet, hash delle transazioni) con valori inventati, lasciando intatti date, monete, quantità e causali.

Un testo da cui partire:

```
Devo scrivere una configurazione JSON per importare un file CSV nel programma Giacenze Crypto.
Ti allego lo schema JSON della configurazione, il manuale, una configurazione ufficiale simile
e l'inizio del mio CSV.
Regole:
- Usa solo le chiavi e i valori ammessi dallo schema, niente chiavi inventate.
- Gli indici delle colonne contano da 0.
- Ogni causale presente nel CSV deve comparire in mappaCausali, con un tipo interno dello schema
  scritto esattamente come lì (in maiuscolo).
- causaliChiuse contiene tipi interni, non causali del CSV.
- Se il file ha la sola data senza ora usa oraSeSoloData.
- Se usi chiavi indicate come "Dalla versione X" imposta versioneMinimaApp.
- Non mettere "centralizzato": true.
Alla fine spiegami, causale per causale, che tipo hai scelto e perché, così posso controllarlo.
```

Chiedere la spiegazione causale per causale è la parte più utile, perché la scelta del tipo interno decide come il movimento viene trattato nel calcolo.

Prima di usare il file:

- Se lo apri con un editor che legge gli schemi JSON (per esempio Visual Studio Code), aggiungi come prima riga `"$schema": "https://lucapasselli.github.io/GiacenzeCrypto/schema/configurazione-import.schema.json",` e l'editor sottolinea chiavi sconosciute, valori non ammessi e numeri scritti come testo. Il programma ignora questa chiave. Le versioni fino alla 1.0.65.13 la elencano fra le opzioni non riconosciute del resoconto, senza altri effetti.

- Prova la configurazione prima di importare: vedi Provare una configurazione. Il resoconto della prova si può incollare all'assistente, che vede cosa è uscito e cosa correggere.
- Dopo l'importazione controlla nella tabella dei movimenti qualche riga per ogni causale: verso, monete, quantità e tipo.

Gli errori più frequenti delle configurazioni scritte da un'IA, tutti segnalati dallo schema tranne gli ultimi due (che la prova della configurazione fa vedere nei movimenti):

- Un tipo interno inventato (`DEPOSITO CRYPTO`) o in minuscolo (`reward`). Il programma non lo rifiuta: tratta la riga come un movimento generico e ne decide la natura solo dalle monete, quindi una ricompensa diventa un deposito da classificare.
- Causali del CSV scritte in `causaliChiuse`, che invece vuole i tipi interni.
- Un indice di colonna scritto fra virgolette (`"3"`) o una chiave che non esiste (`"importo"` dentro `colonne`).
- Un `formatoData` senza l'ora per un file che ha solo la data, che fa scartare tutte le righe. Vedi `oraSeSoloData`.
- Indici contati da 1 invece che da 0, che lo schema non può vedere: la data finisce nella colonna sbagliata.
- Quantità sempre positive senza `causaliUscita`, così i prelievi diventano depositi.

Provare una configurazione

Disponibile dalla versione 1.0.66

Nella finestra *Importazioni* il pulsante **Prova configurazione** legge un CSV con una configurazione e mostra cosa produrrebbe, **senza importare nulla**. Si può provare la configurazione selezionata nelle tendine oppure un file JSON qualunque, anche fuori dalle cartelle del programma, per esempio appena scritto con un'IA.

La finestra della prova ha quattro schede:

- **Movimenti**: quelli che l'importazione creerebbe, con la causale del CSV da cui nasce ciascuno.
- **Riepilogo per causale**: per ogni causale, o gruppo di causali che il programma unisce in un solo movimento, il tipo interno usato e quali movimenti ne escono. È il punto più rapido per vedere una ricompensa finita fra i depositi o uno scambio spezzato in due movimenti.
- **Saldi per moneta**: entrate, uscite e saldo di ogni moneta, da confrontare con quello che mostra l'exchange.
- **Righe scartate**: le righe che non producono movimenti, con il motivo.

In alto la prova segnala anche le opzioni non riconosciute e i valori che non sono tipi interni. Dopo aver corretto il JSON, **Ripeti la prova** rilegge configurazione e CSV. **Copia resoconto** copia tutto in testo semplice, pronto da incollare in un chatbot. Il resoconto contiene le righe scartate così come sono nel CSV: controlla che non ci siano dati che non vuoi condividere.

La prova non cerca i prezzi (i controvalori sono solo quelli scritti nel CSV) e non fa quello che l'importazione fa sull'archivio dopo aver scritto i movimenti: l'abbinamento degli `SCAMBIO DIFFERITO`, i giroconti di `girocontoFiat` e il controllo dei movimenti già presenti.

Condividere e scaricare configurazioni

Disponibile dalla versione 1.0.66

Una configurazione che funziona può servire ad altri utenti con lo stesso exchange. Il programma ha un catalogo di configurazioni condivise dalla community.

Scaricare. Nella finestra *Importazioni* il pulsante **Configurazioni della community** apre il catalogo. **Scarica** salva le configurazioni scelte nella cartella `config/community/` della cartella di lavoro, e da quel momento compaiono nelle tendine con il prefisso *Community:*. Prima di salvare un file il programma ne verifica l'impronta e ripete tutti i controlli di sicurezza. Le configurazioni della community sono controllate dall'autore del programma prima della pubblicazione, ma nessuno può provarle su ogni file reale: **prima di importare, prova la configurazione sul tuo CSV** e controlla i movimenti.

Condividere. Nella finestra della prova il pulsante **Condividi con la community** si attiva solo dopo una prova senza righe scartate, senza opzioni o tipi sconosciuti e con almeno un movimento. La finestra di invio mostra il file così com'è e chiede il consenso alla pubblicazione con licenza MIT. Insieme al file parte l'esito della prova, cioè quanti movimenti escono da ogni causale, **ma nessuna riga del tuo CSV**. La configurazione diventa scaricabile solo dopo il controllo dell'autore del programma. Dalla finestra del catalogo, **Le mie proposte** permette di ritirare una configurazione inviata. Cosa viene conservato è descritto nell'informativa sulla privacy .

Una configurazione condivisa deve rispettare alcune regole in più di una personale:

- Niente `centralizzato`.
- `separatore` solo `,` `;` tabulazione o `|`.
- Nessun carattere `<` nei testi, salvo i commenti.
- Nessun dato personale riconoscibile: indirizzi di wallet, hash di transazione, email, IBAN.
- Al massimo 64 KB.

Struttura generale

GIACENZE CRYPTO

```
{
"$schema": "https://lucapasselli.github.io/GiacenzeCrypto/schema/configurazione-import.schema.json",
"nomeExchange": "...",
"fontePrezzoPreferita": "",
"fornitore": "...",
"estrazione": "...",
"descrizione": "",
"versione": "",
"versioneMinimaApp": "",
"nomeWallet": "...",
"testing": false,
"separatore": ",",
"separatoreValoreMoneta": "",
"coppia": { ... },
"encoding": "UTF-8",
"righeIntestazione": 1,
"rigaIntestazione": 1,
"autoDetectColonne": false,
"mappaAutoDetect": { ... },
"formatoData": "yyyy-MM-dd HH:mm:ss",
"oraSeSoloData": "12:00:00",
"fuso": "UTC",
"fusoDaIntestazione": "",
"fusoPerPeriodo": [ ... ],
"colonneCalcolate": [ ... ],
"regoleRiga": [ ... ],
"consolidaRigheStessaData": false,
"tolleranzaSecondiConsolidamento": 2,
"causaliDifferite": [],
"causaliConsolidaPerGiorno": [],
"causaliAnnullamento": { ... },
"minutiScambioDifferito": 15,
"colonne": { ... },
"raggruppamentoPerCausale": { ... },
"colonneControvalore": { ... },
"contropartitaDaNota": { ... },
"mappaCausali": { ... },
"causalePerNota": [ ... ],
"tipoSeCrypto": { ... },
"tipoSeSola": [ ... ],
"causaliChiuse": [ ... ],
"causaliUscita": [ ... ],
"causaliEntrata": [ ... ],
"versoForzato": "",
"causaliScambiaGambe": [ ... ],
"gambaDoppiaConSegnoSuUscita": false,
"ricostruisciLordoSeFeeSuMonetaUscita": false,
"ricostruisciLordoSeFeeSuMonetaEntrata": false,
"rimuoviCaseSensitive": false,
"rimuoviDaNomeMoneta": [ ... ],
"rinominaMonete": { ... },
"walletPerCausale": { ... },
"walletSpeculativePerCausale": { ... },
"girocontoFiat": { ... },
"causaliDerivati": { ... },
"causaliAllertaDerivati": [ ... ],
"campiExtra": { ... },
"centralizzato": false,
"noteRevisione": { ... },
"causaliAPI": { ... },
"separatoreCausale": ".",
"causaliUppercase": false
}
```

Le chiavi che cominciano con `_` sono commenti, ammessi in qualunque punto del file (`"_nota": "..."`).

1. Intestazione

nomeExchange

Tipo: stringa | **Default:** "" (stringa vuota)

Nome dell'exchange o della piattaforma sorgente. Se lasciato vuoto, il programma chiede interattivamente il nome in fase di importazione.

```
"nomeExchange": "Binance"
```

Non cambiarlo dopo aver importato dei movimenti. Il nome finisce nel campo Exchange del movimento e il programma lo usa in due modi. Per riconoscere i duplicati quando reimporti lo stesso file, e per decidere a quale gruppo wallet appartiene il movimento nel calcolo fiscale. Se lo modifichi, i movimenti già in archivio non vengono più riconosciuti, quindi un nuovo import li duplica, e passano a un altro gruppo wallet, a meno che tu non li riunisca con un alias. `nomeWallet` invece non ha effetti fiscali e può essere cambiato liberamente.

fontePrezzoPreferita

Tipo: stringa | **Default:** "" (nessuna preferenza)

Exchange da preferire, fra quelli già scaricati, quando si cerca il prezzo di un movimento importato da questa configurazione — lo stesso id usato da CCXT, minuscolo (`"binance"`, `"okx"`, `"cryptocom"`, `"bybit"`, `"coinbase"`, `"bitstamp"`, `"kucoin"`). Non limita da chi si scarica: tutti gli exchange configurati vengono comunque interrogati al momento del download dei prezzi; questo campo sceglie solo quale dei prezzi già in cache usare quando più di uno è disponibile per lo stesso movimento. Si applica solo ai movimenti che non hanno già un prezzo o un controvalore nel CSV stesso (colonne `prezzo` e `valoreEuro` entrambe assenti/vuote), e solo al momento dell'import — le rivalorizzazioni fiscali successive non lo consultano.

```
"fontePrezzoPreferita": "binance"
```

nomeWallet

Tipo: stringa | **Default:** "Principale"

Nome del wallet di destinazione. Può essere sovrascritto causale per causale tramite walletPerCausale.

```
"nomeWallet": "Spot"
```

testing

Tipo: booleano | **Default:** false

Se true, segnala in fase di importazione che il file di configurazione è in fase di test e l'importazione potrebbe non essere affidabile.

```
"testing": true
```

fornitore e estrazione

Tipo: stringa | **Default:** "" (stringa vuota)

Decidono **dove compare la configurazione** nelle due tendine della finestra di importazione: `fornitore` è la voce della prima tendina (l'exchange o il servizio), `estrazione` quella della seconda (quale export di quel fornitore si sta caricando).

Vanno indicati solo quando non si ricavano da soli: per un exchange basta `nomeExchange`, e per i formati che contengono i movimenti di piattaforme diverse (CoinTracking, Tatax) basta la parola nel nome del file. Se mancano entrambi si usa il nome del file di configurazione. `estrazione` va scritta **senza ripetere** il nome del fornitore, che è già nella prima tendina.

```
"fornitore": "OKX",
"estrazione": "Funding"
```

`descrizione`

Tipo: stringa | **Default:** "" (nessun suggerimento)

Testo libero che descrive la configurazione: da dove si scarica il file, cosa copre, quali sono i limiti noti. Non cambia in nessun modo l'importazione. Viene mostrato come suggerimento (tooltip) quando passi il mouse sulla voce nella finestra di importazione. Per andare a capo e limitare la larghezza conviene scriverlo in HTML, come fanno le configurazioni ufficiali.

```
"descrizione": "<html><div style='width:380px'>Export CSV dei movimenti di conto. Gli interessi
giornalieri sono sommati in un solo movimento.</div></html>"
```

`versione`

Tipo: stringa | **Default:** ""

Numero di revisione del file (per esempio `"1.002"`). Il programma non lo usa per importare: serve a chi mantiene la configurazione per capire quale revisione è installata, e conviene aumentarlo a ogni modifica se condividi il file con altri.

`versioneMinimaApp`

Tipo: stringa | **Default:** "" (nessun vincolo)

Versione minima del programma che sa interpretare questa configurazione, per esempio `"1.0.64.06"`. Va indicata quando il file usa una funzione del formato aggiunta di recente (tra queste `separatoreValoreMoneta`, `causaliScambiaGambe`, `versoForzato`). Un'installazione più vecchia non propone la configurazione nella finestra di importazione, invece di proporla e poi leggerla male senza accorgersene. Se il vincolo non serve, lascia il campo vuoto.

Dalla versione 1.0.66, se una configurazione distribuita con il programma passa a richiedere una versione più recente di quella installata, il programma continua a usare la copia che ha già, invece di sostituirla con quella nuova e smettere di proporla.

Un'opzione che il programma non conosce, per un errore di battitura nel nome o perché aggiunta da una versione più recente, viene ignorata, e il resoconto dell'importazione la elenca fra le *opzioni della configurazione non riconosciute*. Le chiavi che cominciano con _ sono commenti e non vengono mai segnalate.

```
"versioneMinimaApp": "1.0.64.06"
```

2. Struttura fisica del CSV

`separatore`

Tipo: stringa | **Default:** ","

Carattere che divide le colonne nel CSV.

Valore	Quando usarlo
","	Standard internazionale, default.
";"	Export da Excel con impostazioni europee.
"\t"	File TSV (tab-separated).
"|"	Caso raro, usato da alcuni exchange.

```
"separatore": ";"
```

Numeri e separatore. Nei numeri il programma accetta sia il punto sia la virgola come separatore decimale (`0.5` e `0,5` valgono uguale), ma non il separatore delle migliaia. In un CSV separato da virgole un numero con la virgola va fra virgolette (`"0,5"`): senza, la virgola divide il campo in due. Questi casi finiscono fra le righe scartate, con il motivo:

- `PIU' COLONNE DELL'INTESTAZIONE`: la riga ha più colonne piene della riga di intestazione (`rigaIntestazione`), di solito per un `12,5` senza virgolette. Le colonne vuote in fondo alla riga non contano.
- `NUMERO SPEZZATO DAL SEPARATORE`: la riga ha più campi dell'intestazione, contando anche quelli vuoti in fondo, e una quantità, una commissione, un controvalore o un prezzo è un numero intero seguito da un campo di sole cifre, come `12` e `5` per `12,5`.
- `NUMERO NON LEGGIBILE`: una quantità, una commissione, un controvalore o un prezzo con delle cifre che però non è un numero, per esempio `1.234,56` con il punto delle migliaia. Un valore senza cifre, come `--` o `N/A`, vale come cella vuota.

Un caso non si riconosce: una riga scritta a mano che ha esattamente le colonne dell'intestazione perché manca l'ultima, per esempio `...,BTC,12,5` senza il campo finale. Il `5` prende il posto dell'ultima colonna e la quantità diventa 12. Negli export veri un numero intero seguito da una colonna di sole cifre è normale (una commissione `0` seguita da un'altra colonna `0`), quindi scartare queste righe toglierebbe anche righe giuste. In un CSV scritto o corretto a mano conviene mettere i numeri con la virgola fra virgolette, oppure usare il punto.

`encoding`

Tipo: stringa | **Default:** "UTF-8"

Usa "UTF-8" nella maggior parte dei casi. Se i caratteri speciali risultano corrotti, prova "ISO-8859-1" oppure "Windows-1252".

`righeIntestazione`

Tipo: numero intero | **Default:** 1

Quante righe iniziali del CSV vanno saltate prima dei dati. Se il CSV non ha intestazione, imposta 0.

`rigaIntestazione`

Tipo: numero intero | **Default:** 1

Quale riga (indice 1-based) contiene i nomi delle colonne. Con autoDetectColonne a true le colonne si cercano lì. In ogni caso il programma ne conta le colonne, e scarta le righe di dati che ne hanno di più (vedi Numeri e

separatore). Se quella riga non arriva fino alle colonne lette (per esempio un titolo al posto dell'intestazione), il controllo non si fa.

`autoDetectColonne`

Tipo: booleano | **Default:** false

Se true, legge i nomi delle colonne dalla riga di intestazione e assegna automaticamente gli indici tramite mappaAutoDetect.

`mappaAutoDetect`

Tipo: oggetto chiave-valore

Mappa il nome dell'intestazione CSV al nome del campo logico. Valori validi: data, causale, moneta, quantita, segno, valoreEuro, monetaFee, quantitaFee, idTransazione, idGruppo, wallet. (`prezzo`, `monetaUscita`, `quantitaUscita`, `causale2`, `causale3` non sono riconosciuti dall'auto-detect: vanno indicati a mano in `colonne` se servono.)

```
"autoDetectColonne": true,
"mappaAutoDetect": {
"Timestamp": "data",
"Type": "causale",
"Asset": "moneta",
"Amount": "quantita",
"Fee Amount": "quantitaFee",
"Fee Asset": "monetaFee",
"TxID": "idTransazione"
}
```

`formatoData`

Tipo: stringa | **Default:** "yyyy-MM-dd HH:mm:ss"

Formato della data/ora nel CSV, con i token Java di DateTimeFormatter.

Token	Significato	Esempio
yyyy	Anno a 4 cifre	2025
yy	Anno a 2 cifre	25
MM	Mese a 2 cifre	06
dd	Giorno a 2 cifre	17
HH	Ora 0-23	18
mm	Minuti	39
ss	Secondi	10

```
"formatoData": "dd.MM.yyyy HH:mm:ss"
"formatoData": "MM/dd/yy HH:mm"
```

Il testo fisso va fra apici singoli e le parti che a volte mancano fra parentesi quadre:

```
"formatoData": "yyyy-MM-dd'T'HH:mm:ss.SSS'Z'"
"formatoData": "yyyy-MM-dd HH:mm:ss[.SSS]"
```

Il formato deve comprendere l'ora. Con `"yyyy-MM-dd"` una data come `2025-01-10` non viene letta e la riga viene scartata con il motivo `DATA NON PARSABILE`. Per un file con il solo giorno usa un formato con l'ora e `oraSeSoloData`.

`oraSeSoloData`

Tipo: stringa `HH:mm:ss` | **Default:** assente | **Disponibile dalla versione 1.0.64.09**

Ora da aggiungere alle date che nel file hanno solo il giorno. Funziona sia per un file che ha sempre la sola data sia per un file che mescola righe con e senza ora (lo storico Simple Earn di Binance). L'ora è sempre in **UTC**, qualunque sia il `fuso` del file: `"12:00:00"` tiene il movimento nello stesso giorno anche in ora italiana, con o senza ora legale.

```
"formatoData": "yyyy-MM-dd HH:mm:ss",
"oraSeSoloData": "12:00:00"
```

Le righe dello stesso giorno ricevono la stessa ora: se non sono parti dello stesso movimento lascia `consolidaRigheStessaData` a false, altrimenti verrebbero unite.

`fuso`

Tipo: stringa | **Default:** "UTC"

Fuso orario delle date nel CSV. Vengono convertite nel fuso locale dell'app.

```
"fuso": "UTC"
"fuso": "Europe/Rome"
"fuso": "UTC+2"
```

Con un punto interrogativo finale (`"UTC?"`) vince il fuso scritto nel **nome del file**, se c'è (Binance mette nel nome dell'export il fuso scelto sul sito, per esempio `(UTC+2)`). Lasciato vuoto (`""`), il programma usa il fuso del nome del file e, se nemmeno lì c'è, lo chiede al momento dell'importazione.

`fusoDaIntestazione`

Tipo: stringa | **Default:** assente | **Disponibile dalla versione 1.0.64.10**

Etichetta con cui il file dichiara il proprio fuso in una delle righe di intestazione, per esempio `Time Zone: UTC+8` negli export di OKX, dove il fuso è quello scelto dall'utente. Se la trova, il fuso del file vince su `fuso`. Se il file non la contiene (export vecchi) resta `fuso`.

```
"fusoDaIntestazione": "Time Zone:"
```

`fusoPerPeriodo`

Tipo: array di `{ da, a, fuso }` | **Default:** [] | **Disponibile dalla versione 1.0.65.02**

Per i file che hanno cambiato fuso nel tempo (il file dei depositi e prelievi di Nexo Pro è in UTC nel 2023 e in ora locale nel 2025). Ogni voce dà il fuso delle righe con la data compresa fra `da` e `a` (estremi compresi, formato `AAAA-MM-GG`), e vince su tutto il resto. I periodi non possono sovrapporsi.

```
"fusoPerPeriodo": [
  { "da": "2023-01-01", "a": "2023-12-31", "fuso": "UTC" },
  { "da": "2024-01-01", "a": "2025-12-31", "fuso": "Europe/Rome" }
]
```

Non aggiungerlo a una configurazione con cui hai già importato: sposta l'orario dei movimenti di quel periodo, e un nuovo import non li riconosce più come già presenti.

3. Colonne del CSV

Mappa i campi logici agli indici numerici delle colonne del CSV, contando da 0. Usa -1 per indicare che una colonna non è presente.

```
"colonne": {
"data": 0,
"causale": 1,
"moneta": 2,
"quantita": 3,
"segno": -1,
"valoreEuro": -1,
"prezzo": -1,
"monetaFee": -1,
"quantitaFee": -1,
"idTransazione": -1,
"wallet": -1,
"monetaUscita": -1,
"quantitaUscita":-1,
"note": -1,
"causale2": -1,
"causale3": -1
}
```

Come trovare l'indice di una colonna

Apri il CSV, guarda la riga di intestazione e conta le colonne partendo da 0.

```
"Timestamp","Tipo","Asset","Quantita","Valore EUR"
0 1 2 3 4
```

Quindi: "data": 0, "causale": 1, "moneta": 2 e così via.

Campi disponibili

Campo	Descrizione
data	Obbligatorio - data e ora del movimento.
causale	Tipo di operazione nel CSV (Deposit, Trade, Staking...).
moneta	Simbolo del token (BTC, ETH...).
quantita	Quantità del movimento (negativa=uscita, positiva=entrata).
segno	Colonna separata che indica il segno (+ o -).
valoreEuro	Controvalore in euro già calcolato nel CSV.
prezzo	Prezzo unitario del token al momento dell'operazione.
monetaFee	Simbolo della moneta usata per la commissione.
quantitaFee	Quantità della commissione pagata.
idTransazione	ID univoco per raggruppare righe correlate dello stesso trade.
wallet	Nome del wallet, se variabile riga per riga.

Campo	Descrizione
monetaUscita	Moneta in uscita (CSV con entrata e uscita sulla stessa riga).
quantitaUscita	Quantita in uscita (CSV con entrata e uscita sulla stessa riga).
note	Colonna di testo libero da riportare nel campo Note del movimento. Serve anche a `causalePerNota` (vedi sezione 4).

Importo e moneta nella stessa cella

Tipo: stringa | **Default:** "" (importo e moneta sono due colonne distinte)

Alcuni export scrivono importo e simbolo insieme, per esempio `400 AME`. In questo caso indica lo stesso indice di colonna sia per `quantita` sia per `moneta` (e, se serve, per `quantitaUscita`/`monetaUscita` e `quantitaFee`/`monetaFee`), e in `separatoreValoreMoneta` il carattere che divide i due pezzi. Quando le due colonne coincidono il programma separa il numero dal simbolo usando quel carattere.

```
"separatoreValoreMoneta": " ",
"colonne": { "quantita": 5, "moneta": 5, "quantitaFee": 6, "monetaFee": 6 }
```

Coppia di trading in una sola cella

Tipo: oggetto `{ colonna, separatore, base, quote }` | **Default:** assente

Alcuni export di trading non hanno colonne per le monete e riportano solo la coppia, per esempio `KCS-ETH`. Il blocco `coppia` dice dove si trova la coppia e come dividerla. Il programma aggiunge a ogni riga due **colonne virtuali** alle posizioni `base` e `quote`, che poi usi nelle `colonne` come una qualsiasi altra colonna (di solito `base` come `moneta` e `quote` come `monetaUscita`).

Campo	Descrizione
`colonna`	Indice della colonna che contiene la coppia.
`separatore`	Carattere che divide base e quota. Default `-`. Non può essere vuoto.
`base`	Indice della prima colonna virtuale. Scegli un numero **oltre l'ultima colonna vera** del CSV.
`quote`	Indice della seconda colonna virtuale, stessa regola.

Una riga la cui cella non contiene il separatore viene scartata e segnalata come `COPPIA NON VALIDA`.

```
"coppia": { "colonna": 3, "separatore": "-", "base": 16, "quote": 17 },
"colonne": { "moneta": 16, "quantita": 7, "monetaUscita": 17, "quantitaUscita": 8 },
"causaliScambiaGambe": ["SELL"]
```

L'esempio è quello di KuCoin: la coppia sta nella colonna 3, le colonne virtuali 16 e 17 vengono dopo l'ultima colonna vera, e le vendite (`SELL`) hanno i ruoli di entrata e uscita invertiti rispetto agli acquisti (vedi `causaliScambiaGambe`).

`importoQuote`

Tipo: oggetto `{ colonna, quantita, prezzo }` dentro `coppia` | **Disponibile dalla versione 1.0.65.01**

Per gli export che riportano quantità e prezzo ma non l'importo della seconda moneta (Nexo Pro). Il programma crea un'altra colonna virtuale, alla posizione `colonna`, con il prodotto esatto della colonna `quantita` per la colonna `prezzo`. La cella resta vuota se uno dei due manca o non è un numero (un ordine annullato).

```
"coppia": { "colonna": 2, "separatore": "/", "base": 14, "quote": 15,
            "importoQuote": { "colonna": 16, "quantita": 9, "prezzo": 6 } },
"colonne": { "moneta": 14, "quantita": 9, "monetaUscita": 15, "quantitaUscita": 16 }
```

Trasformare le righe prima di leggerle

Due strumenti semplici per i file che non hanno già tutto quello che serve in una colonna. Si applicano a ogni riga subito dopo la lettura, in quest'ordine: `coppia`, `colonneCalcolate`, `regoleRiga`. Ognuno può quindi usare le colonne prodotte da quelli prima.

`colonneCalcolate`

Tipo: array di `{ colonna, operazione, a, b }` | **Default:** [] | **Disponibile dalla versione 1.0.65.02**

Crea una colonna virtuale alla posizione `colonna` con il risultato di un'operazione fra le colonne `a` e `b`: `prodotto`, `quoziente` (a diviso b), `somma`, `differenza` (a meno b). Le voci si applicano in ordine, quindi una può usare il risultato di quelle prima. La cella resta vuota se un operando manca o non è un numero, o se il divisore è zero. Scegli per `colonna` un numero oltre l'ultima colonna vera del CSV.

```
"colonneCalcolate": [
  { "colonna": 12, "operazione": "prodotto", "a": 5, "b": 6 }
],
"colonne": { "valoreEuro": 12 }
```

`regoleRiga`

Tipo: array di regole | **Default:** [] | **Disponibile dalla versione 1.0.65.02**

Ogni regola ha **una** condizione su una colonna e **una** azione. Le regole si valutano tutte, in ordine, e ognuna vede le modifiche di quelle prima. Una regola che scarta la riga ferma le successive.

Campo	Descrizione
`colonna`	Colonna da controllare.
`uguale`	Condizione: la cella è uguale a questo testo, senza distinzione fra maiuscole e minuscole.
`contiene`	Condizione: la cella contiene questo testo, senza distinzione fra maiuscole e minuscole.
`vuota`	Condizione: `true` se la cella è vuota, `false` se non lo è.
`imposta`	Azione: `{ "colonna": n, "valore": "testo" }` scrive un valore fisso nella colonna `n`. Non può scrivere la colonna della data.
`scarta`	Azione: `true` scarta la riga, che compare nel resoconto con il motivo `REGOLA DI RIGA`.

```
"regoleRiga": [
  { "colonna": 3, "uguale": "Transfer", "imposta": { "colonna": 3, "valore": "Deposit" } },
  { "colonna": 4, "vuota": true, "scarta": true }
]
```

Non è un linguaggio di regole: niente condizioni combinate e niente valori presi da altre colonne. Se servono, è un caso da segnalare.

Causale composita

Alcuni CSV distribuiscono il tipo di operazione su più colonne. Si possono combinare fino a 3 colonne in una causale composita concatenate da separatoreCausale.

Parametro	Default	Descrizione
causale2 (in colonne)	-1	Indice della seconda colonna causale.
causale3 (in colonne)	-1	Indice della terza colonna causale.
separatoreCausale	"."	Carattere di giunzione tra le parti.
causaliUppercase	false	Se true converte ogni parte in maiuscolo prima di concatenare.

Esempio: causale=1 ("Trade"), causale2=2 ("Buy"), separatoreCausale="." produce la chiave "Trade.Buy" da usare in mappaCausali.

```
"colonne": { "causale": 1, "causale2": 2 },
"separatoreCausale": ".",
"mappaCausali": { "Trade.Buy": "SCAMBIO CRYPTO-CRYPTO" }
```

Entrata e uscita sulla stessa riga

Alcuni CSV (es. Koinly, CoinTracking) riportano l'intero scambio su una sola riga. In quel caso moneta/quantita = lato entrata, monetaUscita/quantitaUscita = lato uscita.

```
"colonne": {
"data": 8, "causale": 0,
"moneta": 2, "quantita": 1,
"monetaUscita": 4, "quantitaUscita": 3,
"monetaFee": 6, "quantitaFee": 5
}
```

`gambaDoppiaConSegnoSuUscita`

Tipo: booleano | **Default:** false

Per gli export (tipico Nexo) in cui **ogni** riga ha due colonne moneta, ma solo quando le monete sono diverse si tratta di uno scambio. Quando la moneta di uscita coincide con quella principale (interessi, versamenti, prelievi...) la riga è un movimento a gamba singola e il verso sta nel segno della colonna di uscita. Con `true` il programma applica questa regola, altrimenti un accredito di interessi `NEXO ? NEXO` diventerebbe un finto scambio. Se l'importo principale è zero si usa quello della colonna di uscita.

`causaliScambiaGambe`

Tipo: array di causali CSV originali | **Default:** [] (nessuna)

Per le righe che portano già entrambe le gambe, scambia il ruolo di entrata e uscita delle due colonne prima di ogni altra elaborazione. Serve quando l'export usa colonne dal ruolo fisso (per esempio "Deal amount" e "Total") ma la direzione reale dipende dalla causale: con `Buy` entra "Deal amount" ed esce "Total", con `Sell` accade il contrario. Elenca qui le causali per cui le colonne vanno invertite. Se moneta e quantità condividono la stessa cella (`separatoreValoreMoneta`) simbolo e importo si scambiano insieme.

```
"causaliScambiaGambe": ["Sell"]
```

`idGruppo` (nella sezione colonne)

Tipo: numero intero | **Default:** -1 (assente)

Colonna su cui raggruppare le righe **quando non coincide con quella dell'identificativo**. I due ruoli sono distinti: `idTransazione` dice *chi è* la riga — finisce nel movimento ed è ciò su cui lavora il controllo dei duplicati — mentre `idGruppo` dice *con chi sta*.

Nell'export di trading di OKX, ad esempio, ogni gamba e ogni esecuzione parziale hanno un `id` diverso ma condividono l'`Order id`: usare il primo anche per raggruppare spezzerebbe ogni scambio in gambe isolate. Quando `idGruppo` non è indicata si raggruppa come sempre sull'identificativo, quindi le configurazioni che non la usano non cambiano comportamento.

Quando `idGruppo` è presente, il file viene inoltre **ordinato** per quel campo (e poi per `idTransazione`) prima dell'importazione, perché il raggruppamento lavora su righe consecutive e gli export intercalano le gambe di ordini diversi.

```
"colonne": { "idTransazione": 0, "idGruppo": 1 }
```

`raggruppamentoPerCausale`

Tipo: oggetto causale CSV → indice colonna | **Default:** {} (nessun override)

Sovrascrive la colonna di raggruppamento **solo per le causali elencate**. Serve quando le gambe di uno stesso movimento condividono una colonna diversa da `idGruppo`/`idTransazione` e i loro timestamp non sono identici.

Caso tipico (Coinbase): le due gambe di un `Convert` hanno la stessa identica stringa nella colonna `note` (`"Converted 20 ALEPH to 5 ZETA"`) ma sull'export a volte cadono a 1-2 secondi di distanza; raggruppando sul timestamp esatto verrebbero spezzate in un deposito e un prelievo separati. Raggruppando sulla colonna `note` (indice 10) si ricompongono. Va abbinato a `consolidaRigheStessaData` + `causaliDifferite` + una `tolleranzaSecondiConsolidamento` sufficiente. Una gamba con valore diverso in quella colonna apre comunque un gruppo nuovo, quindi movimenti distinti non si mescolano.

```
"raggruppamentoPerCausale": { "Convert": 10 },
"consolidaRigheStessaData": true,
"tolleranzaSecondiConsolidamento": 60,
"causaliDifferite": ["Convert"]
```

4. Mappatura delle causali

`mappaCausali`

Tipo: oggetto chiave-valore

Traduce le causali originali del CSV nelle tipologie interne dell'applicazione. Ogni causale deve avere una voce qui, altrimenti la riga viene scartata. La ricerca è case insensitive per default.

```
"mappaCausali": {
"Deposit": "TRASFERIMENTO-CRYPTO",
"Withdrawal": "TRASFERIMENTO-CRYPTO",
"Trade": "SCAMBIO CRYPTO-CRYPTO",
"Staking Rewards": "STAKING REWARDS",
"Commission": "COMMISSIONI",
"earn":"EARN",
"cashback":"CASHBACK"
}
```

Per ignorare righe senza contarle come scarti, mappa su "IGNORA" (vale anche "NON CONSIDERARE"):

```
"mappaCausali": { "Internal Transfer": "IGNORA" }
```

Tipologie interne che puoi usare come valore:

Valore	Quando usarlo
`SCAMBIO CRYPTO-CRYPTO`	Scambio fra due monete. Anche `DUST-CONVERSION` per la conversione dei saldi minimi.
`ACQUISTO CRYPTO` / `VENDITA CRYPTO`	Acquisto o vendita di crypto contro valuta FIAT.
`DEPOSITO FIAT` / `PRELIEVO FIAT`	Versamento o prelievo di valuta FIAT.
`TRASFERIMENTO-CRYPTO`	Deposito o prelievo di crypto. Il verso lo decide il segno della quantità, per questo **non esistono** tipologie separate per deposito e prelievo.
`TRASFERIMENTO-CRYPTO-INTERNO`	Spostamento fra comparti dello stesso exchange. Non genera plusvalenze.
`SCAMBIO DIFFERITO`	Metà di uno scambio che compare come due righe indipendenti (vedi sezione 6).
`REWARD`, `ALTRE-REWARD`, `EARN`, `STAKING REWARDS`, `CASHBACK`, `AIRDROP`	Ricompense di vario tipo. Una ricompensa con quantità negativa diventa il rimborso di quella ricompensa.
`COMMISSIONI`	Commissione pagata come movimento a sé. Con quantità positiva diventa un cashback.
`SPESA`	Come `TRASFERIMENTO-CRYPTO`: la natura la decidono le monete e il verso. La usano le configurazioni CoinTracking.
`CASHOUT O SIMILARE`	Prelievo già classificato come cashout o similare (vedi il manuale *Classificazione dei movimenti*).
`RETTIFICA GIACENZA`	Prelievo già classificato come rettifica di giacenza.
`DEPOSITO A COSTO 0`	Deposito già classificato come deposito a costo zero.
`IGNORA`	Riga da saltare senza segnalarla come scartata. Vale anche `NON CONSIDERARE`.

Sono ammessi anche alcuni sinonimi, con lo stesso effetto: `STAKING REWARD` (come `STAKING REWARDS`), `COMMISSIONE` (come `COMMISSIONI`), `TRASFERIMENTO INTERNO` e `TRANSAZIONE TECNICA INTERNA` (come `TRASFERIMENTO-CRYPTO-INTERNO`). L'elenco completo è nello schema JSON.

Scrivi il tipo esattamente come in tabella, in maiuscolo. Il programma non rifiuta un tipo che non conosce: un valore come `reward` o `DEPOSITO CRYPTO` produce un movimento generico, la cui natura viene decisa solo dalle monete e dal verso, quindi una ricompensa diventa un deposito da classificare a mano. Lo schema JSON lo segnala, e dalla versione 1.0.66 lo segnalano anche la prova della configurazione e il resoconto dell'importazione, fra i *tipi di movimento non riconosciuti*.

Una causale non presente in `mappaCausali` (e non coperta da `causalePerNota` o da `contropartitaDaNota`) fa scartare la riga, che compare fra i movimenti sconosciuti del resoconto con il testo `CAUSALE SCONOSCIUTA`. Per altri esempi apri le configurazioni ufficiali nella cartella `config/import/`.

`causalePerNota`

Tipo: array di regole `{ causale, notaContiene, tipo }` | **Default:** [] (nessuna regola)

Riclassifica una riga in base al **testo libero della colonna** `note`, quando la causale del CSV da sola non basta a distinguere la natura del movimento. Richiede che `colonne.note` sia impostata.

Ogni regola:

Campo	Obbligatorio	Descrizione
`causale`	no	Causale CSV che la riga deve avere. Se omessa, la regola vale per qualsiasi causale.
`notaContiene`	sì	Sottostringa cercata nella colonna `note` (confronto **case-insensitive**).
`tipo`	sì	Tipologia interna da assegnare (le stesse usabili in `mappaCausali`: `EARN`, `REWARD`, `AIRDROP`, `STAKING REWARDS`, `COMMISSIONI`, ...).

Le regole si valutano **in ordine: vince la prima che combacia**. Se nessuna regola combacia si usa la normale `mappaCausali`.

Caso reale (Coinbase): i premi arrivano tutti come causale `Receive` e si riconoscono solo dalla nota.

```
"colonne": { "note": 10 },
"causalePerNota": [
  { "causale": "Receive", "notaContiene": "from Coinbase Earn",     "tipo": "EARN" },
  { "causale": "Receive", "notaContiene": "from Coinbase Referral", "tipo": "REWARD" },
  { "causale": "Receive", "notaContiene": "airdrop",                "tipo": "AIRDROP" }
]
```

`tipoSeCrypto`

Tipo: oggetto causale CSV → tipologia interna | **Default:** {} (nessuna voce)

Dà a una causale un tipo diverso **quando la moneta della riga non è una valuta FIAT** (euro, dollaro e le altre valute riconosciute dal programma). Le righe della stessa causale in valuta FIAT restano col tipo della `mappaCausali`. Si valuta dopo `causalePerNota` e prima della `mappaCausali`. Disponibile dalla versione 1.0.65.07: le versioni precedenti ignorano la chiave e usano per tutte le righe il tipo della `mappaCausali`.

Caso reale (carta Bybit): `Purchase` è la spesa in euro, ma anche la cripto ceduta quando la spesa è coperta da USDC. In quel caso il file scrive tre righe a un secondo l'una dall'altra: `Purchase` con gli USDC ceduti, `Coin Purchase` con gli euro ricevuti e `Purchase` con gli stessi euro spesi. Con la voce qui sotto le prime due diventano una vendita USDC → EUR e la terza un prelievo fiat, come le spese in euro.

```
"tipoSeCrypto": { "Purchase": "SCAMBIO CRYPTO-CRYPTO" },
"mappaCausali": {
  "Purchase":      "TRASFERIMENTO-CRYPTO",
  "Coin Purchase": "SCAMBIO CRYPTO-CRYPTO"
},
"causaliDifferite": ["Purchase", "Coin Purchase"],
"tolleranzaSecondiConsolidamento": 1
```

`tipoSeSola`

Tipo: array di regole `{ causale, con, tipo }` | **Default:** [] | **Disponibile dalla versione 1.0.64.07**

Tipo di ripiego per una causale che di solito arriva insieme a un'altra ma a volte arriva da sola. Se nel gruppo di righe della causale `causale` c'è almeno una delle causali elencate in `con`, la riga prende il tipo della `mappaCausali`. Altrimenti prende `tipo`.

Caso reale (Bybit): il token ricevuto da un Launchpad (`airdropAssetIncrease`) arriva un secondo dopo il BIT impegnato (`commitmentForLaunchpad`), e insieme formano uno scambio. Ma lo stesso token può arrivare anche da solo, e allora è un airdrop.

```
"tipoSeSola": [
  { "causale": "airdropAssetIncrease", "con": ["commitmentForLaunchpad"], "tipo": "AIRDROP" }
]
```

`causaliChiuse`

Tipo: array di stringhe (tipologie interne)

Tipologie interne che devono essere trattate come movimenti singoli anche se condividono l'ID transazione con altre righe. Usa per staking, cashback, commissioni, depositi e prelievi. Si indica la **tipologia interna** (il valore di `mappaCausali`), non la causale del CSV. In vecchie configurazioni la chiave si chiama `movimentoChiuso`, che il programma accetta ancora come sinonimo.

```
"causaliChiuse": [
"TRASFERIMENTO-CRYPTO",
"STAKING REWARDS","CASHBACK","COMMISSIONI"
]
```

5. Gestione del segno

Per default il segno viene letto dalla quantità (negativa = uscita, positiva = entrata). Se la quantità è sempre positiva e il verso si deduce dalla causale, usa causaliUscita e causaliEntrata.

`causaliUscita`

Tipo: array di causali CSV originali

Le righe con queste causali avranno la quantità forzata negativa.

```
"causaliUscita": ["Withdrawal","Trade Sell","Fee"]
```

`causaliEntrata`

Tipo: array di causali CSV originali

Le righe con queste causali avranno la quantità forzata positiva.

```
"causaliEntrata": ["Deposit","Trade Buy","Staking Rewards","Cashback"]
```

`versoForzato`

Tipo: stringa `"ENTRATA"` o `"USCITA"` | **Default:** "" (nessuna forzatura)

Forza il verso di **ogni** riga del file, qualunque sia la causale. Serve per gli export divisi per direzione, come i file dei soli depositi o dei soli prelievi di Gate.io, in cui non c'è nessuna colonna causale affidabile da cui dedurre il verso. A differenza di `causaliUscita`/`causaliEntrata` non dipende dal testo della causale. Qualunque altro valore viene ignorato.

```
"versoForzato": "ENTRATA"
```

Esempio pratico

```
"Data","Tipo","Moneta","Quantita"
"2025-01-15","Deposito","BTC","0.5"
"2025-01-16","Prelievo","ETH","1.2"
"causaliUscita": ["Prelievo"],
"causaliEntrata": ["Deposito","Staking","Cashback"]
```

6. Consolidamento righe correlate

Alcuni exchange suddividono una singola operazione in più righe CSV (es. riga per moneta venduta + riga per moneta acquistata). Le opzioni seguenti permettono di raggrupparle.

`consolidaRigheStessaData`

Tipo: booleano | **Default:** false

- false: ogni riga è un movimento indipendente.
- true: le righe con stesso ID transazione o timestamp vicino vengono raggruppate.

```
"consolidaRigheStessaData": true
```

`tolleranzaSecondiConsolidamento`

Tipo: numero intero | **Default:** 2

Usato solo se `consolidaRigheStessaData` è true. Quanti secondi di differenza sono tollerati tra due righe perché vengano considerate parte della stessa operazione.

```
"tolleranzaSecondiConsolidamento": 5
```

`causaliDifferite`

Tipo: array di causali CSV originali

Se specificato, la tolleranza temporale viene applicata solo ai gruppi che contengono almeno una riga con queste causali; per le altre viene richiesto timestamp identico.

```
"causaliDifferite": ["Trade","Swap"]
```

`causaliConsolidaPerGiorno`

Tipo: array di causali CSV originali | **Default:** [] (nessuna)

Somma le righe con queste causali in **un solo movimento per giorno, moneta e causale**. È pensato per gli export che accreditano micro-interessi molte volte al giorno (un export Bitget ne conteneva 28.000): senza la somma l'archivio si riempie di decine di migliaia di movimenti minuscoli. Il movimento risultante prende la data del primo accredito del giorno e le altre colonne dalla prima riga, con la sola quantità sostituita dalla

somma. La commissione della prima riga viene azzerata, perché non va attribuita all'intero giorno. Il giorno è quello del fuso del tuo sistema. La somma avviene prima del raggruppamento delle righe.

```
"causaliConsolidaPerGiorno": ["Interest"]
```

`causaliAnnullamento`

Tipo: oggetto causale dell'annullamento → causale annullata | **Default:** {} (nessuna)

Alcuni export riportano un prelievo e, più tardi, il suo annullamento con la moneta che torna indietro (Bybit: `Withdrawal` e poi `Withdrawal Canceled`). Per ogni riga di annullamento il programma cerca, fra le righe precedenti dello stesso file, la più recente con la causale annullata, la stessa moneta e la stessa quantità di segno opposto: se la trova, **nessuna delle due righe viene importata**, perché la moneta non è mai uscita dal wallet. Se non la trova (il prelievo sta in un export precedente, oppure l'importo restituito è diverso) l'annullamento viene importato secondo `mappaCausali`, quindi va mappato anche lì. Lo stesso avviene se il prelievo è già nell'archivio da un'importazione precedente: il prelievo viene scartato come già esistente e l'annullamento importato, così la giacenza resta corretta. L'abbinamento avviene prima di ogni altra elaborazione delle righe.

```
"causaliAnnullamento": { "Withdrawal Canceled": "Withdrawal" }
```

La causale `SCAMBIO DIFFERITO` e `minutiScambioDifferito`

Alcune operazioni (l'Auto-Invest di Binance, un recupero fondi, una redistribuzione di token) mostrano nel CSV due righe indipendenti — un prelievo e un deposito, spesso di monete diverse — che in realtà sono le due metà dello stesso scambio, avvenuto "dietro le quinte" dell'exchange. Mappando la causale su `SCAMBIO DIFFERITO` (invece che su `TRASFERIMENTO-CRYPTO`) quella riga entra anche nella ricerca automatica di abbinamento a fine import: se, fra tutte le righe `SCAMBIO DIFFERITO` di questa importazione, un prelievo trova un deposito entro `minutiScambioDifferito` minuti e con un controvalore che non si scosta di oltre il 10%, i due movimenti vengono trasformati in un vero scambio (trasferimento verso una piattaforma fittizia, scambio, trasferimento di ritorno) invece di restare due movimenti scollegati.

`SCAMBIO DIFFERITO` va sempre aggiunta anche a `causaliChiuse` (è un movimento a gamba singola, come `TRASFERIMENTO-CRYPTO`).

Tipo: numero intero (minuti) | **Default:** 15

```
"minutiScambioDifferito": 15
```

Il valore di default (15 minuti) è quello storicamente usato per Binance; un altro exchange, con tempi di regolamento diversi, può richiederne uno diverso.

`idTransazione` (nella sezione colonne)

Se il CSV ha una colonna con un ID univoco per transazione, indicarla consente di raggruppare righe anche se i timestamp differiscono oltre la tolleranza.

```
"colonne": { "idTransazione": 5 }
```

7. Gestione commissioni

`ricostruisciLordoSeFeeSuMonetaUscita`

Tipo: booleano | **Default:** false

Controlla se, quando la commissione è nella stessa moneta dell'uscita, il sistema deve ricostruire il lordo del movimento principale.

- true: ricostruisce il lordo e genera anche il movimento commissione separato.
- false: non modifica il movimento principale, genera solo il movimento commissione.

Esempio: uscita -500 USDC e fee 1 USDC. Con true: movimento principale -499 USDC + movimento commissione -1 USDC. Usalo quando la quantità nel CSV è già comprensiva della fee (riga unica).

```
"ricostruisciLordoSeFeeSuMonetaUscita": true
```

`ricostruisciLordoSeFeeSuMonetaEntrata`

Tipo: booleano | **Default:** false

Controlla se, quando la commissione è nella stessa moneta dell'entrata, il sistema deve ricostruire il lordo del movimento principale.

- true: ricostruisce il lordo e genera anche il movimento commissione separato.
- false: non modifica il movimento principale, genera solo il movimento commissione.

Esempio: entrata 500 USDC e fee 1 USDC. Con true: movimento principale 501 USDC + movimento commissione -1 USDC. Usalo quando la quantità nel CSV è al netto della fee (riga unica).

```
"ricostruisciLordoSeFeeSuMonetaEntrata": true
```

`colonneControvalore`

Tipo: oggetto | **Default:** assente

Per gli export in cui la colonna del totale **comprende sia la commissione sia lo spread** (tipico di Coinbase retail: `Total (inclusive of fees and/or spread)`). Per le cripto-attività la **commissione non è deducibile** dal costo di carico (art. 68 c. 9-bis TUIR), ma lo **spread sì** — non è una commissione. Con questo blocco, per le sole causali indicate, il controvalore usato come costo di carico diventa `totale ? commissione` (lo spread resta dentro), invece della colonna `valoreEuro`.

Campo	Descrizione
`totale`	Indice colonna del totale comprensivo di commissione e spread.
`commissione`	Indice colonna della commissione. Conta solo se è un importo **positivo** (alcune righe dust contengono un rapporto di spread con segno: viene ignorato).
`valuta`	Indice colonna con la valuta di `totale`/`commissione` (es. la colonna "Price Currency" = `EUR`). Usata per la gamba FIAT sintetica e per il movimento commissione.
`causali`	Causali CSV per cui applicare il ricalcolo `totale ? commissione`.
`causaliConMovimentoCommissione`	Sottoinsieme di `causali`: righe che portano la **sola gamba crypto in entrata**. Per queste viene **sintetizzata la gamba FIAT in uscita** di importo `totale ? commissione` (l'operazione diventa un vero

GIACENZE CRYPTO

Campo	Descrizione
	acquisto FIAT→crypto invece di un semplice deposito) e la commissione esce come **movimento** `COMMISSIONI` **a sé** nella valuta indicata.
`noteAcquistoDaSaldo`	Elenco di testi (confronto senza distinzione fra maiuscole e minuscole) che, nella colonna `note`, indicano un acquisto pagato **dal saldo** dell'exchange (per Coinbase `"EUR Wallet"`). Vale solo per le causali di `causaliConMovimentoCommissione`. Se la nota non è vuota e **non** contiene nessuno di questi testi, l'acquisto è stato pagato con uno strumento esterno al saldo (carta, Apple Pay, Google Pay, bonifico diretto): in quel caso non si sintetizzano né la gamba FIAT né il movimento commissione, e resta il solo deposito crypto, caricato a costo pieno (`totale ? commissione`) senza muovere euro. Nota vuota oppure elenco vuoto: comportamento normale.

Per le causali in `causali` ma **non** in `causaliConMovimentoCommissione` (es. gli scambi crypto/crypto, dove le monete sono già nette e la commissione è espressa solo in euro) viene corretto solo il controvalore, senza movimento commissione aggiuntivo.

```
"colonne": { "valoreEuro": 7 },
"colonneControvalore": {
  "totale": 8,
  "commissione": 9,
  "valuta": 5,
  "causali": ["Buy", "Convert"],
  "causaliConMovimentoCommissione": ["Buy"],
  "noteAcquistoDaSaldo": ["EUR Wallet"]
}
```

`contropartitaDaNota`

Tipo: oggetto | **Default:** assente | **Disponibile dalla versione 1.0.66**

Per le righe che portano **una sola moneta** mentre l'altra moneta dello scambio e il suo importo stanno solo nel testo della colonna `note`. È il caso degli ordini *Advanced Trade* e delle vendite semplici di Coinbase, con note come `Bought 0.5 FORTH for 3.015 EUR on FORTH-EUR at 6 EUR/FORTH` oppure `Sold 50 USDC for 46.5 EUR`. Il programma legge dalla nota la seconda moneta (EUR, USDC...) e l'importo, separa la commissione e la emette come **movimento** `COMMISSIONI` nella stessa moneta, come per gli acquisti di `colonneControvalore`. La commissione è la differenza fra l'importo della nota e quantità × prezzo (il numero dopo `at`), o fra l'importo della nota e la colonna `valoreEuro` quando la nota non ha il prezzo, cosa possibile solo se la seconda moneta è l'euro.

Campo	Descrizione
`causali`	Oggetto causale CSV → tipo di movimento, come in `mappaCausali`. Le causali elencate qui **non** vanno messe anche in `mappaCausali`. Il tipo è solo un'etichetta: acquisto, vendita o scambio crypto-crypto li decidono le due monete.
`commissioneEuro`	Indice colonna con la commissione già convertita in euro: diventa il valore del movimento commissione. Facoltativo.

La nota deve tornare con la riga: verbo (`Bought` con quantità positiva, `Sold` con quantità negativa), quantità e moneta uguali a quelle delle colonne. Una nota che non torna, o che non ha questa forma, fa scartare la riga con il motivo `NOTA SENZA CONTROPARTITA`: il programma non prova a indovinare.

Le righe di queste causali sono sempre trattate come movimenti a sé, anche quando più righe cadono nello stesso secondo (un ordine eseguito in più parti). Le versioni del programma precedenti alla 1.0.66 ignorano la chiave e scartano queste righe come causali sconosciute, che è il motivo per cui il tipo sta qui e non in

`mappaCausali`.

```
"contropartitaDaNota": {
  "causali": {
    "Advanced Trade Buy":  "ACQUISTO CRYPTO",
    "Advanced Trade Sell": "VENDITA CRYPTO",
    "Sell":                "VENDITA CRYPTO"
  },
  "commissioneEuro": 9
}
```

8. Pulizia e normalizzazione nomi moneta

`rimuoviDaNomeMoneta`

Tipo: array di stringhe

Rimuove testi indesiderati dal nome del token. Supporta la sintassi ? per troncare tutto ciò che viene dopo o prima.

Sintassi	Effetto su BTC.STAKING@CRYPTO.COM	Risultato
.STAKING?	Rimuove .STAKING e tutto quello che segue	BTC
?.STAKING	Rimuove .STAKING e tutto quello che precede	@CRYPTO.COM
.STAKING	Rimuove solo la parola esatta	BTC@CRYPTO.COM

Le regole vengono applicate in sequenza. Prima quelle più ampie (con ?).

```
"rimuoviDaNomeMoneta": [".STAKING?",".EARN?",".LOCKED?","@CRYPTO.COM"]
```

`rimuoviCaseSensitive`

Tipo: booleano | **Default:** false

- false: la ricerca ignora maiuscole e minuscole.
- true: la ricerca e case sensitive esatta.

`rinominaMonete`

Tipo: oggetto chiave-valore

Rinomina un simbolo moneta. La rinomina avviene dopo la pulizia con rimuoviDaNomeMoneta.

```
"rinominaMonete": { "IOTA": "MIOTA", "LUNA2": "LUNA", "WBTC": "BTC" }
```

9. Wallet per causale e giroconti fra wallet

`walletPerCausale`

Tipo: oggetto chiave-valore

Permette di assegnare un wallet diverso da nomeWallet per specifiche causali. La chiave è la causale originale del CSV.

```
"walletPerCausale": {
"Staking Rewards": "Staking", "Earn": "Earn", "Lockup": "Locked"
}
```

`walletSpecularePerCausale`

Tipo: oggetto chiave-valore

Per le causali indicate viene creato, oltre al movimento normale, un **secondo movimento uguale e contrario** sul wallet indicato: se la riga porta 100 USDT in uscita dal wallet principale, ne viene generata l'entrata di 100 USDT sul wallet della gamba speculare. La chiave è la causale originale del CSV.

Serve per le operazioni che spostano una moneta in un comparto dell'exchange e la restituiscono più tardi, con un rendimento e a volte in un'altra moneta: sono due righe del CSV distanti settimane, che l'importazione non può accoppiare da sola. Le due gambe tengono il saldo esatto su entrambi i lati, e la differenza fra quanto è uscito e quanto è rientrato resta come **giacenza negativa** sul comparto, segnalata fra gli errori e da sistemare a mano registrando la reward corrispondente.

```
"walletSpecularePerCausale": {
"Dual Savings Purchase": "Investimenti", "Dual Savings Settlement": "Investimenti"
}
```

Tre cose da sapere:

- **Il nome dell'exchange non cambia**: entrambe le gambe restano sullo stesso exchange, cambia solo il nome del wallet. È voluto, perché l'exchange determina il gruppo wallet usato per il calcolo fiscale.
- **La causale va mappata su** `TRASFERIMENTO-CRYPTO-INTERNO` e quel valore va aggiunto anche a `causaliChiuse`. Così le due gambe sono spostamenti interni, che non generano plusvalenze, e restano due movimenti distinti anche quando più righe cadono nello stesso secondo.
- **Vale solo per le righe che muovono una moneta sola.** Su una riga che ne muove già due la gamba speculare non viene creata e l'importazione prosegue con il solo movimento normale.

`girocontoFiat`

Tipo: oggetto | **Default:** assente

Per gli **euro** (o un'altra valuta FIAT) spostati fra due wallet tuoi, per esempio da Coinbase ad Coinbase Pro. Nei CSV sono un prelievo FIAT su un wallet e un deposito FIAT sull'altro, indistinguibili da un bonifico dalla banca: chi conta i depositi FIAT per sapere quanto ha versato conta due volte gli stessi soldi, e il quadro RW li legge come apporto e prelievo. Con questo blocco, alla fine di ogni importazione, il programma cerca nell'**intero archivio** le coppie e le trasforma in un normale trasferimento fra wallet. La categoria resta deposito/prelievo FIAT, così il saldo si sposta davvero da un wallet all'altro.

Campo	Descrizione
`controparte`	`nomeExchange` dell'altro wallet del giroconto.
`causali`	Causali CSV dei giroconti su **questo** wallet.
`causaliControparte`	Causali CSV dei giroconti sul wallet controparte.
`secondiTolleranza`	Scarto massimo in secondi fra le due gambe. Default 60.

```
"girocontoFiat": {
  "controparte": "Coinbase",
  "causali": ["deposit", "withdrawal"],
  "causaliControparte": ["Exchange Deposit", "Exchange Withdrawal", "Pro Deposit", "Pro Withdrawal"],
  "secondiTolleranza": 60
}
```

Tre cose da sapere:

- **Dichiara la regola in entrambe le configurazioni**, ognuna con le parti invertite. Così funziona in qualunque ordine importi i due file e basta reimportare uno dei due per sistemare anche i movimenti già in archivio.
- **Una coppia viene riconosciuta solo se entrambe le causali CSV sono fra quelle dichiarate.** È questo che tiene fuori un normale bonifico dalla banca (`Deposit` sul wallet retail), che non è un giroconto.
- Importo uguale, segno opposto, stessa valuta e scarto di tempo entro la tolleranza. Una riga senza controparte resta com'è. Lo stesso abbinamento si può fare a mano dalla classificazione del movimento.

10. Derivati

Il programma **non calcola i redditi da derivati** (futures, perpetui, opzioni, Dual Investment, art. 67 c-quater TUIR): li tratta come cripto-attività, e questo non è corretto sul piano fiscale. Le due chiavi seguenti servono a farli riconoscere e ad avvisare l'utente.

`causaliDerivati`

Tipo: oggetto causale CSV → tipo | **Default:** {}

Per ogni causale elencata, i movimenti che nascono da quella riga ricevono nel campo Derivato un tipo che li identifica. Il tipo non cambia il calcolo, serve solo a riconoscerli e filtrarli. Una causale presente qui fa comparire anche l'avviso di fine importazione e l'avvertenza nelle stampe dei quadri W/RW e T/RT.

I tipi usati dalle configurazioni ufficiali sono `PNL` (risultato realizzato), `FUNDING`, `BONUS`, `RIMBORSO_COMMISSIONI`, `CONSEGNA`, `DUAL` e `COMMISSIONE`.

```
"causaliDerivati": {
"funding": "FUNDING", "trade": "PNL", "feeRefund": "RIMBORSO_COMMISSIONI"
}
```

`causaliAllertaDerivati`

Tipo: array di causali CSV originali | **Default:** []

Fa comparire l'avviso sui derivati a fine importazione per le causali elencate, senza marcare i movimenti. Le causali di `causaliDerivati` entrano automaticamente anche qui. Le configurazioni ufficiali ripetono le stesse causali in **entrambe** le chiavi, perché le versioni del programma più vecchie ignorano `causaliDerivati` e leggono solo questa. Se scrivi una configurazione solo per la tua installazione basta `causaliDerivati`.

```
"causaliAllertaDerivati": ["Dual Savings Purchase"]
```

11. Campi extra

`campiExtra`

Tipo: oggetto indiceMovimento -> indiceColonnaCSV

Copia il contenuto di una colonna CSV in un campo specifico del movimento. Funzione avanzata e raramente necessaria.

```
"campiExtra": { "7": 9 }
```

12. Centralizzato

`centralizzato`

Tipo: booleano | **Default:** false

Indica che questo file è gestito centralmente dal repository ufficiale. All'avvio del programma, quando vengono controllati i file di configurazione da GitHub, se un file locale ha centralizzato: true e non è più presente nel repository, viene automaticamente eliminato.

I file creati localmente dall'utente non devono avere centralizzato: true, altrimenti potrebbero essere cancellati involontariamente.

Un file senza `centralizzato: true` è per il programma una **configurazione personalizzata**: nella finestra di importazione compare con il prefisso *Personalizzata:* davanti al nome dell'estrazione, in corsivo nell'elenco e dopo le voci ufficiali dello stesso exchange. Il suggerimento che compare passando il mouse sulla voce indica il nome del file e la cartella in cui si trova. Serve a riconoscerla anche quando dichiara lo stesso `nomeExchange` di un exchange già previsto dal programma, e finisce quindi sotto quell'exchange accanto alle voci ufficiali.

```
"centralizzato": true
```

`noteRevisione`, `causaliAPI` e `$schema`

Tre chiavi che non cambiano la lettura del CSV:

- `noteRevisione` (oggetto causale → testo): appunti di chi mantiene il file, per esempio le causali da ricontrollare. Il programma non li usa.
- `causaliAPI` (oggetto codice → causale, **dalla versione 1.0.65.05**): serve solo alle configurazioni lette anche dall'importazione via API di Bitget, per tradurre i codici di tipo dell'API nelle causali del CSV. In una configurazione per un file non serve.
- `$schema` (testo): l'indirizzo dello schema JSON, per gli editor che controllano il file mentre lo scrivi (vedi Scrivere la configurazione con un'IA). Il programma la ignora. Le versioni che non la conoscono la elencano fra le opzioni non riconosciute del resoconto, senza altri effetti.

Esempi completi

Gli esempi seguenti sono verificati da un test automatico del programma: ogni CSV viene letto con la sua configurazione e deve produrre i movimenti indicati sotto il JSON, senza righe scartate.

Esempio 1: Una riga per ogni moneta, con il segno - es. Binance

Ogni riga muove una sola moneta e il segno della quantità dice il verso. Le righe di uno stesso scambio hanno lo stesso istante e vengono unite.

CSV:

```
"User_ID","UTC_Time","Account","Operation","Coin","Change","Remark"
"10000001","2025-03-10 14:22:01","Spot","Transaction Buy","BTC","0.01",""
"10000001","2025-03-10 14:22:01","Spot","Transaction Spend","USDT","-620.50",""
"10000001","2025-03-10 14:22:01","Spot","Transaction Fee","BTC","-0.00001",""
"10000001","2025-03-12 09:05:44","Spot","Deposit","ETH","0.5",""
```

JSON:

```
{
"nomeExchange": "Binance", "nomeWallet": "Principale",
"separatore": ",", "formatoData": "yyyy-MM-dd HH:mm:ss", "fuso": "UTC",
"colonne": { "data": 1, "causale": 3, "moneta": 4, "quantita": 5 },
"consolidaRigheStessaData": true, "tolleranzaSecondiConsolidamento": 1,
"mappaCausali": {
"Transaction Buy": "SCAMBIO CRYPTO-CRYPTO",
"Transaction Spend": "SCAMBIO CRYPTO-CRYPTO",
"Transaction Fee": "COMMISSIONI",
"Deposit": "TRASFERIMENTO-CRYPTO"
},
"causaliChiuse": ["TRASFERIMENTO-CRYPTO", "COMMISSIONI"]
}
```

Risultato: uno scambio USDT → BTC, una commissione in BTC e un deposito di ETH.

Esempio 2: Riga singola per movimento - es. Tatax

CSV:

```
"Data","Tipo","Asset","Quantita"
"2025-09-15 10:00:00","Deposito","BTC","0.5"
"2025-09-16 11:30:00","Staking","BTC.STAKING@CRYPTO.COM","0.0001"
"2025-09-17 09:00:00","Prelievo","ETH","0.1"
```

JSON:

```
{
"nomeExchange": "Crypto.com", "nomeWallet": "Principale",
"separatore": ",", "formatoData": "yyyy-MM-dd HH:mm:ss", "fuso": "UTC",
"consolidaRigheStessaData": false,
"colonne": { "data": 0, "causale": 1, "moneta": 2, "quantita": 3 },
"mappaCausali": {
"Deposito": "TRASFERIMENTO-CRYPTO", "Prelievo": "TRASFERIMENTO-CRYPTO",
"Staking": "STAKING REWARDS"
},
"causaliChiuse": ["TRASFERIMENTO-CRYPTO","STAKING REWARDS"],
"causaliEntrata": ["Deposito","Staking"],
"causaliUscita": ["Prelievo"],
"rimuoviDaNomeMoneta": [".STAKING?",".EARN?","@CRYPTO.COM"],
"rimuoviCaseSensitive": false
}
```

Risultato: un deposito di BTC, una ricompensa di staking in BTC (il nome `BTC.STAKING@CRYPTO.COM` viene ripulito) e un prelievo di ETH.

Esempio 3: Scambio su riga singola - es. CoinTracking

CSV:

```
"Tipo","Acquisto","Cur.","Vendita","Cur.","Fee","Cur.Fee","Exchange","Data"
"Operazione","2132","CRO","487.04","USDC","1.18","USDC","Crypto.com","17.09.2025 18:39:10"
"Deposito","89.4","CRO","","","","","Crypto.com","28.08.2025 07:38:42"
"Prelievo","","","24.32","USDC","","","Crypto.com","03.09.2025 18:11:09"
```

JSON:

```
{
"nomeExchange": "Crypto.com Exchange", "nomeWallet": "Principale",
"separatore": ",", "formatoData": "dd.MM.yyyy HH:mm:ss", "fuso": "UTC",
"colonne": {
"data": 8, "causale": 0, "moneta": 2, "quantita": 1,
"monetaUscita": 4, "quantitaUscita": 3, "quantitaFee": 5, "monetaFee": 6
},
"mappaCausali": {
"Deposito": "TRASFERIMENTO-CRYPTO", "Prelievo": "TRASFERIMENTO-CRYPTO",
"Operazione": "SCAMBIO CRYPTO-CRYPTO"
},
"ricostruisciLordoSeFeeSuMonetaUscita": false,
"ricostruisciLordoSeFeeSuMonetaEntrata": true
}
```

Risultato: uno scambio USDC → CRO con la sua commissione in USDC, un deposito di CRO e un prelievo di USDC. La quantità della colonna `quantitaUscita` conta sempre come uscita, con o senza il segno meno.

Esempio 4: Causale composita su più colonne

CSV:

```
"Date","Category","SubType","Amount","Currency"
"2025-01-10 08:30:00","Trade","Buy","0.005","BTC"
"2025-01-10 08:30:00","Trade","Sell","200","USDT"
"2025-01-11 00:00:00","Earn","Staking","0.0001","ETH"
```

Le quantità sono tutte positive: il verso lo dà la causale, quindi la vendita va in `causaliUscita`, che confronta la causale composita (`Trade.Sell`).

JSON:

```
{
"nomeExchange": "Exchange XYZ", "separatore": ",",
"formatoData": "yyyy-MM-dd HH:mm:ss", "fuso": "UTC",
"consolidaRigheStessaData": true,
"colonne": { "data": 0, "moneta": 4, "quantita": 3, "causale": 1, "causale2": 2 },
"separatoreCausale": ".", "causaliUppercase": false,
"mappaCausali": {
"Trade.Buy": "SCAMBIO CRYPTO-CRYPTO",
"Trade.Sell": "SCAMBIO CRYPTO-CRYPTO",
"Earn.Staking": "STAKING REWARDS"
},
"causaliChiuse": ["STAKING REWARDS"],
"causaliUscita": ["Trade.Sell"]
}
```

Risultato: uno scambio USDT → BTC e una ricompensa di staking in ETH.

Esempio 5: Solo la data, senza l'ora

CSV, separato da punto e virgola e con la virgola decimale tipica degli export in italiano:

```
Data;Operazione;Moneta;Importo
15/01/2025;Interessi;USDC;0,52
16/01/2025;Interessi;USDC;0,51
20/01/2025;Prelievo;USDC;-100
```

JSON:

```
{
"nomeExchange": "Exchange XYZ", "nomeWallet": "Rendita",
"separatore": ";", "formatoData": "dd/MM/yyyy HH:mm:ss", "oraSeSoloData": "12:00:00",
"colonne": { "data": 0, "causale": 1, "moneta": 2, "quantita": 3 },
"mappaCausali": {
"Interessi": "EARN",
"Prelievo": "TRASFERIMENTO-CRYPTO"
},
"causaliChiuse": ["EARN", "TRASFERIMENTO-CRYPTO"]
}
```

Il formato della data comprende l'ora anche se il file non ce l'ha: è `oraSeSoloData` ad aggiungerla, alle 12 UTC, così il giorno resta lo stesso anche in ora italiana. Con `"formatoData": "dd/MM/yyyy"` tutte le righe verrebbero scartate.

Risultato: due ricompense e un prelievo di USDC.

Checklist rapida

- Apri il CSV e conta le colonne partendo da 0.
- Identifica data, causale, moneta e quantità; compila la sezione colonne.
- Se la data non ha l'ora, usa un `formatoData` con l'ora e `oraSeSoloData`.
- Elenca tutte le causali presenti nel CSV e compila mappaCausali.
- Se le quantità sono sempre positive, compila causaliUscita e causaliEntrata.
- Se ogni riga è un movimento indipendente usa consolidaRigheStessaData: false; se più righe appartengono allo stesso trade usa true.
- Se i nomi moneta contengono suffissi, compila rimuoviDaNomeMoneta.
- Se ci sono commissioni, mappa monetaFee e quantitaFee; se la quantità è già comprensiva della fee abilita il flag ricostruisciLordo appropriato.
- Se la colonna del totale include commissione **e** spread (Coinbase), usa colonneControvalore per scorporare la sola commissione dal costo di carico.
- Se un acquisto può essere pagato anche con carta o bonifico diretto (Coinbase), usa `noteAcquistoDaSaldo` per non creare euro in uscita che non sono mai passati dal saldo.
- Se la natura del movimento è scritta solo nel testo libero di una colonna (es. "from Coinbase Earn"), mappa colonne.note e aggiungi le regole in causalePerNota.
- Se le gambe di uno stesso movimento condividono una colonna diversa da idGruppo ma non il timestamp esatto, usa raggruppamentoPerCausale.
- Se alcuni movimenti devono andare su wallet separati, compila walletPerCausale.
- Se una causale sposta una moneta in un comparto dell'exchange e una seconda causale la restituisce settimane dopo (depositi vincolati, Dual Investment), usa walletSpeculativePerCausale.
- Se le intestazioni CSV variano di versione in versione, usa autoDetectColonne con mappaAutoDetect.
- Se importo e moneta stanno nella stessa cella (`400 AME`), usa `separatoreValoreMoneta`. Se c'è solo la coppia di trading (`KCS-ETH`), usa `coppia`.

- Se il file contiene solo depositi o solo prelievi e non ha una colonna causale utile, usa `versoForzato`.
- Se il verso delle due gambe dipende dalla causale (Buy/Sell sulle stesse colonne), usa `causaliScambiaGambe`. Se ogni riga ha due monete ma solo a volte è uno scambio, usa `gambaDoppiaConSegnoSuUscita`.
- Se l'export accredita micro-interessi decine di volte al giorno, somma le righe con `causaliConsolidaPerGiorno`.
- Se gli euro passano fra due tuoi wallet (Coinbase e Coinbase Pro), dichiara `girocontoFiat` in entrambe le configurazioni.
- Se il file contiene derivati, elenca le causali in `causaliDerivati` per avere l'avviso e il marcatore.
- Prima di condividere il file, compila `descrizione` e, se usa funzioni recenti, `versioneMinimaApp`.
- Se il tipo di operazione è su più colonne, usa causale2/causale3 con separatoreCausale.
- Se il CSV non riporta né prezzo né controvalore e vuoi che venga usato il prezzo di un exchange specifico fra quelli già scaricati, compila fontePrezzoPreferita.

Dove mettere il file

Nella cartella di lavoro del programma si trovano due cartelle di configurazione:

- `config/import/` — è la cartella attuale, sincronizzata con il repository ufficiale: qui arrivano le configurazioni distribuite con il programma;
- `ImportConfig/` — la cartella storica, mantenuta per compatibilità con le installazioni precedenti. Da qui vengono letti soltanto i file **non** marcati `"centralizzato": true`, cioè quelli scritti dall'utente.

Un file JSON messo in una di queste due cartelle compare nella finestra di importazione insieme agli import nativi, ordinato per fornitore ed estrazione. Un file scritto dall'utente si riconosce dal prefisso *Personalizzata:* (vedi `centralizzato`).

I file creati personalmente vanno lasciati **senza** `centralizzato` (o con `"centralizzato": false`): non verranno mai eliminati automaticamente dall'aggiornamento dal repository remoto.

GIACENZE CRYPTO